

WHATSAPP INFRASTRUCTURE GUIDE

בניית WhatsApp Hub עצמאי

מדריך מלא – מאפס ועד API חי שמדבר עם WhatsApp,
כולל אבטחה, חשיפה לאינטרנט ואינטגרציה לכל פלטפורמה

⌚ זמן ביצוע: ~45 דקות · €3.79 לחודש · יכולת בלתי מוגבלת

מחבר

Noam Nissan

תאריך

מאי 2026

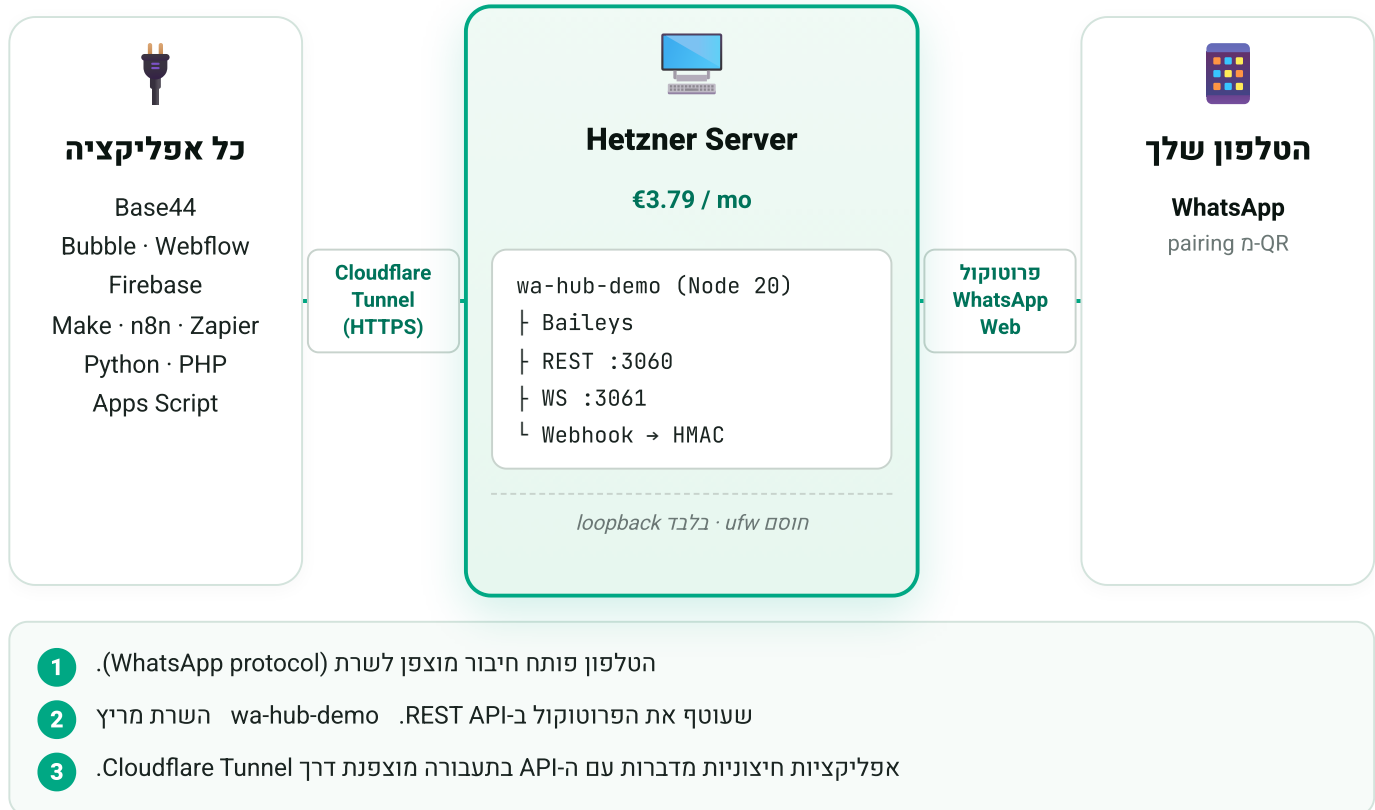
github.com/noamnissan/wa-hub-demo

בניית WhatsApp Hub עצמאי – מדריך מלא

מה תקבל בסוף המדריך הזה: REST API מאובטח שמדבר עם WhatsApp, רץ על שרת שלך, חשוף לאינטרנט בצורה הגיונית, ומדבר עם כל פלטפורמה שאתה רוצה – Base44, Bubble, Firebase, Make, Python, או כל דבר שיוזע HTTP.

זמן ביצוע: ~45 דקות בידיים. **עלות:** €3.79 לחודש (Hetzner CAX11). **רישיון:** קוד מקור: MIT.
github.com/noamnissan/wa-hub-demo

מה אנחנו בונים?



מה זה כן:

- **HTTP API פשוט** מעל פרוטוקול POST. WhatsApp Web, לשליחה, Webhook לקבלה. כלום מסובך.
- **בעלות מלאה** – הקוד שלך, השרת שלך, ה-token שלך, ההודעות שלך.
- **עלות קבועה** – €3.79 לחודש כל החודש, בלי קשר אם שלחת 10 או מיליון הודעות.
- **קוד פתוח** (MIT) – תפרק, תשנה, תפרסם, תמכור.

מה זה לא:

- **לא רשמי מ-Meta**. WhatsApp לא מספקים API ישיר כזה. אנחנו מתחזים ל-Linked Device. זה עובד כי הפרוטוקול גלוי, אבל אם תעבירו לקוד שלכם 10K הודעות ביום של ספאם – WhatsApp יחסום את המספר. השתמשו רק להודעות שמישהו ביקש.
- **לא Multi-tenant out-of-the-box**. מספר אחד = שרת אחד = instance אחד. אם רוצים לתת שירות לכמה לקוחות, כל לקוח מקבל instance נפרד (ראו "צעדים הבאים").
- **לא Plug-and-play של Meta WhatsApp Cloud API**. זה משהו אחר – של Meta, דורש אישור עסקי, וכרוך ב-billing per-message. שני העולמות לא חופפים.

■ למי המדריך הזה:

- **מפתחים** שרוצים שליטה מלאה ב-stack של WhatsApp שלהם
 - **No-code builders** (Base44, Bubble, Webflow) שרוצים לחבר WhatsApp בלי לקנות תוסף
 - **אנשי אוטומציה** (Make, n8n, Zapier) שצריכים אינטגרציה אמינה
 - **חברות קטנות** שמעדיפות לשלם €3.79 לחודש קבוע במקום \$0.05 לכל הודעה
-

רשימת מה שצריך לפני שמתחילים

- ☐ כרטיס אשראי (Hetzner גובה €3.79 בחודש הראשון)
 - ☐ טלפון עם WhatsApp פעיל (להוסיף כ-Linked Device)
 - ☐ חשבון GitHub (חינם, לשכפול הקוד)
 - ☐ חשבון Cloudflare (חינם, אופציונלי – ל-Tunnel)
 - ☐ טרמינל (Mac/Linux: בנוי פנימי. Windows: PowerShell או WSL)
-

שלב 1 – קניית שרת Hetzner

1.1 הרשמה

לכו ל-hetzner.com/cloud ולחצו "Sign Up".

למה Hetzner? היחס מחיר/ביצועים הכי טוב באירופה. €3.79 לחודש = 2 vCPU + 4GB RAM + 40GB SSD + 20TB תעבורה. שווה ערך ל-AWS t4g.medium ב-\$25 לחודש.

1.2 יצירת פרויקט וחיוב

אחרי האימות:

1. הקליקו **"New Project +"** → תנו שם (למשל `wa-hub`)
2. בתפריט שמאל → **"Billing"** → הכניסו פרטי כרטיס
3. בתפריט שמאל → **"Security"** → **"SSH Keys"** → **"Add SSH Key"**

■ 1.3 יצירת מפתח SSH (אם אין לך)

הריצו פקודה אחת בלבד:

על Mac / Linux / WSL:

BASH

```
ssh-keygen -t ed25519 -C "noam-laptop"
```

על Windows PowerShell:

POWERSHELL

```
ssh-keygen -t ed25519 -C "noam-laptop"
```

חשוב: הקלידו את הפקודה **לבד**, בלי להעתיק שורות אחרות אחריה. הפקודה תשאל אתכם 3 שאלות אינטראקטיביות, ואם תדביקו עוד שורה – היא תיכנס בטעות כתשובה לאחת מהן (זה קורה הרבה).

הפקודה תשאל אתכם 3 שאלות. **לכל אחת לחצו Enter** (כל ברירות המחדל בסדר):

1. Enter file in which to save the key → Enter (ברירת מחדל: ~/.ssh/id_ed25519)
2. Enter passphrase (empty for no passphrase) → Enter (בלי סיסמה למפתח)
3. Enter same passphrase again → Enter (אישור)

על passphrase: זה סיסמה שמצפינה את המפתח הפרטי עצמו על הדיסק. עם passphrase תצטרכו להקליד אותה בכל חיבור (אלא אם משתמשים ב-ssh-agent). בלי passphrase נוח יותר. אם המחשב שלכם הוא רק שלכם – Enter פעמיים זה בסדר גמור.

עכשיו **בנפרד** הציגו את המפתח הציבורי:

על Mac / Linux / WSL:

BASH

```
cat ~/.ssh/id_ed25519.pub
```

על Windows PowerShell:

POWERSHELL

```
Get-Content $HOME\.ssh\id_ed25519.pub
```

העתיקו את כל השורה שהודפסה (מתחילה ב- `ssh-ed25519`) → הדביקו ב-Hetzner → תנו שם זיהוי (למשל `laptop`) → שמור.

על C- : זה רק שם שמודבק על המפתח, כמו תווית. רוב המדריכים שמים שם אימייל מהרגל, אבל זה ממש לא חובה ולא מוסיף אבטחה – תן שם שיעזור לך לזהות איזה מפתח זה (`noam-laptop`, `office-mac`, `phone-2026`). המפתח עצמו נוצר מאקראיות קריפטוגרפית של מערכת ההפעלה, לא מהמחרוזת הזאת.

1.4 הזמנת השרת

1. בתפריט שמאל → "Servers" → "Add Server +"

2. **Location:** Falkenstein (גרמניה – הכי קרוב לישראל מבחינת ping, ~60ms). אם אין שם – Nuremberg גם גרמניה ועובד באותה איכות.

3. **Image:** Ubuntu **24.04** LTS (גם LTS 26.04 עובד, אבל היא חדשה מ-2026/04 ופחות בדוקה. 24.04 מספיקה ל-5 שנים קדימה ונבדקה במדריך הזה לעומק).

4. **Type:** בחרו אחד משני אלה (שניהם עובדים זהה – תלוי במה שזמין באתר באותו רגע):

• **CX23** (x86 / Intel-AMD) – €3.99 לחודש · זמין בכל המיקומים

• **CAX11** (ARM / Ampere) – €3.79 לחודש · זמין רק במיקומים מסוימים (כרגע Nuremberg / Helsinki).

אם הטאב "**Arm64 (Ampere)**" קיים – מצוין, אחרת השתמשו ב-CX23.

5. **Networking:** השאירו ברירת מחדל (IPv4 + IPv6)

6. **SSH keys:** סמנו את המפתח שהוספתם

7. **Volumes / Firewalls / Backups:** דלגו (לא צריך)

8. **Name:** `wa-hub-demo` (או מה שתרצו)

9. לחצו "Create & Buy now"

תוך **30 שניות** השרת יהיה מוכן. שימו לב לכתובת ה-IPv4 – תזדקקו לה.

CX23 vs CAX11 – מה ההבדל בפועל? אפס מבחינת הקוד שלנו. שניהם מריצים Node 20 בלי הבדל.

CAX11 קצת יותר יעיל אנרגטית ו-€0.20 זול יותר. CX23 קצת יותר זמין ובמיקומים יותר. **קחו את מה שזמין**

במיקום הקרוב אליכם – זה לא משנה. הזמינות של ARM משתנה מדי פעם כי Hetzner מנהלים מלאי לפי דרישה.

שלב 2 – חיבור SSH ראשון

זה הצעד שהרבה אנשים נתקעים בו בפעם הראשונה. נעבור לאט. **לא תצטרכו להדביק את המפתח שלכם בשום מקום בשלב הזה** – כבר הדבקנו אותו ב-Hetzner בשלב 1.3, וזה מה שמספיק. ה-SSH client במחשב שלכם ימצא לבד את המפתח הפרטי ב- `~/.ssh/id_ed25519` וישתמש בו.

■ 2.1 – מציאת ה-IP של השרת

בפאנל של Hetzner Cloud → השרת שלכם → תחת "IPv4" תראו כתובת כמו `203.0.113.42`. העתיקו אותה – תזדקקו לה בעוד רגע.

■ 2.2 – פתיחת טרמינל

• **Mac**: Cmd+Space → הקלידו `Terminal` → Enter

• **Linux**: Ctrl+Alt+T (תלוי בהפצה)

• **Windows**: Start → הקלידו `PowerShell` → Enter

◦ Windows 10/11 כולל את `ssh` מובנה. לא צריך להתקין כלום.

■ 2.3 – חיבור

הקלידו (החליפו את `203.0.113.42` ב-IP שלכם):

```
BASH
```

```
ssh root@203.0.113.42
```

בפעם הראשונה תופיע הודעה:

```
The authenticity of host '203.0.113.42 (...)' can't be established.  
ED25519 key fingerprint is SHA256:..  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

הקלידו `yes` ו-Enter. זאת רק שאלה חד-פעמית – SSH אומר "לא ראיתי את השרת הזה בעבר, אתה בטוח שזה הוא?"

■ 2.4 – אתם בפנים

אם הכל בסדר, תראו:

```
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-XX-generic x86_64)
root@wa-hub-demo:~#
```

הסימן `#` בסוף השורה אומר שאתם מחוברים כ-`root` לשרת. כל מה שתקלידו מכאן ירוץ **על השרת בגרמניה**, לא על המחשב שלכם.

■ 2.5 – בדיקה זריזה שהשרת חי

BASH

```
lsb_release -d
```

BASH

```
free -h
```

BASH

```
curl -fsSL ifconfig.me; echo
```

הראשונה אומרת מה גרסת Ubuntu, השנייה כמה זיכרון, השלישית מאשרת שיש אינטרנט (היא מדפסה את ה-IP שלכם – אותו אחד שהדבקתם למעלה).

■ 2.6 – אם לא הצליח להתחבר

▼ הודעה: Permission denied (publickey)

זה אומר שה-SSH key לא נמצא או לא תקין:

- ודאו שב-Hetzner סימנתם את ה-SSH key בזמן יצירת השרת. אם שכחתם → צריך לאפס.
- ודאו שיצרתם את המפתח עם `ssh-keygen` (שלב 1.3) ושהמפתח קיים: `ls ~/.ssh/id_ed25519`
- (Mac/Linux) או `ls $HOME\.ssh\id_ed25519` (PowerShell). אם המפתח קיים והתקלה ממשיכה – חכו 30 שניות (לפעמים Hetzner צריך זמן להזריק את המפתח לשרת) ותנסו שוב.

▼ שכחתם להוסיף SSH key בזמן יצירת השרת

מהפאנל של Hetzner:

1. כנסו לשרת → "Rescue" → אפסו את סיסמת root
2. תיכנסו עם הסיסמה שקיבלתם: `ssh root@` והקלידו את הסיסמה
3. הדביקו את המפתח הציבורי שלכם לקובץ `~/.ssh/authorized_keys`:

BASH

```
mkdir -p ~/.ssh  
echo "ssh-ed25519 AAAA... noam-laptop" >> ~/.ssh/authorized_keys  
chmod 700 ~/.ssh  
chmod 600 ~/.ssh/authorized_keys
```

(החליפו את `ssh-ed25519 AAAA...` בתוכן האמיתי של `~/.ssh/id_ed25519.pub` שלכם.)

מהפעם הבאה – תוכלו להיכנס עם המפתח, בלי סיסמה.

▼ הודעה: Connection timed out או Connection refused

- ודאו שהשרת באמת רץ בפאנל של Hetzner (אם הוא בסטטוס "starting" – חכו דקה)
- ודאו שלא העתקתם IP לא נכון (IPv6 נראה אחרת – צריך IPv4)
- אם אתם ברשת מוגבלת (אוניברסיטה, חברה) – פורט 22 לפעמים חסום משם

שלב 3 – בוחרים מסלול: ידני או Claude Code

מכאן יש שתי דרכים לבנות את ה-Hub:

מסלול A – ידני	מסלול B – עם Claude Code
קצב	מהיר
למידה	מבינים את הזרימה
התאמה אישית	קלה (מבקשים מ-Claude)
מומלץ ל-	פעם ראשונה

המלצה לזרימה: התחילו במסלול A (ידני) כדי שהקהל יבין מה קורה. בסוף הראו דמו של B כפיתוי לעתיד.

שלב 4 – מסלול A: התקנה ידנית

כל הפקודות מתחת מורצות **על השרת** (אחרי `ssh root@`). במקום עשרה שלבים קטנים, חילקתי לשלושה בלוקים גדולים שכל אחד עומד בפני עצמו.

■ A.1 – הקשחת השרת

עדכונים + נעילת SSH + firewall + fail2ban בריצה אחת. ~3 דקות.

BASH

```
# עדכונים
apt-get update && apt-get -y dist-upgrade && apt-get -y autoremove

# SSH – בלי סיסמאות, בלי סיסמאות – מפתחות בלבד
sed -i 's/^#*PasswordAuthentication */PasswordAuthentication no/' /etc/ssh/sshd_config
sed -i 's/^#*PermitRootLogin */PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
sshd -t && systemctl reload ssh

# Firewall – רק SSH לאינטרנט פתוח
ufw allow 22/tcp comment 'SSH'
ufw default deny incoming && ufw default allow outgoing
yes | ufw enable

# fail2ban – חוסם IPs שמנסים brute-force
apt-get install -y fail2ban
systemctl enable --now fail2ban
```

מה השגנו: שרת שאי אפשר להיכנס אליו בלי המפתח הפרטי שלך, ובוטים שמנסים ננעלים אוטומטית אחרי 3 ניסיונות. ה-Hub עצמו לא חשוף – נטפל בזה דרך Tunnel בשלב 6.

■ A.2 – התקנת Node, יצירת משתמש שירות, ושכפול הקוד

BASH

```
# Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt-get install -y nodejs

# משתמש שירות (לא root!) אם פולש מצליח להריץ קוד, הוא מקבל הרשאות מוגבלות
useradd --system --shell /usr/sbin/nologin --home-dir /srv/wa-hub-demo --create-home wahub

# שכפול ושיכפול הבילות (כ-wahub, לא כ-root)
cd /srv
sudo -u wahub git clone https://github.com/noamnissan/wa-hub-demo.git
sudo -u wahub bash -c "cd wa-hub-demo && npm ci --omit=dev"
```

`npm ci` מתקין את הגרסאות **המדויקות** מ-`package-lock.json` (לא כמו `npm install` שעלול לעדכן). אותו `setup`, אותן גרסאות, בכל פעם.

BASH

```
# יצירת סודות אקראיים (256 ביט)
HUB_TOKEN=$(openssl rand -hex 32)
WEBHOOK_SECRET=$(openssl rand -hex 32)

# הצג אותם פעם אחת – שמור במנהל סיסמאות
echo "==== שמרו את הערכים האלה ===="
echo "HUB_TOKEN=$HUB_TOKEN"
echo "WEBHOOK_SECRET=$WEBHOOK_SECRET"
echo "===== "

env. כתיבת #
cat > /srv/wa-hub-demo/.env <<EOF
HUB_NAME=wa-hub
HUB_TOKEN=$HUB_TOKEN
WEBHOOK_SECRET=$WEBHOOK_SECRET
HUB_PORT=3060
WS_PORT=3061
RATE_LIMIT_PER_MIN=120
DATA_DIR=/srv/wa-hub-demo/data
LOG_LEVEL=info
EOF
chown wahub:wahub /srv/wa-hub-demo/.env
chmod 600 /srv/wa-hub-demo/.env

data יצירת תיקיית #
mkdir -p /srv/wa-hub-demo/data
chown -R wahub:wahub /srv/wa-hub-demo/data

# התקנה כ- systemd service, מתאושש אוטומטית
install -m 644 /srv/wa-hub-demo/deploy/wa-hub.service /etc/systemd/system/wa-hub.service
systemctl daemon-reload
systemctl enable --now wa-hub.service

# בדיקה אחרונה – אמור להחזיר {"ok":true, "connection":"qr"}
sleep 4
curl -sS http://127.0.0.1:3060/healthz
```

השירות פעיל! `systemd` יחזיר אותו תוך 5 שניות אם הוא נופל, רץ עם `MemoryMax=512M` כדי לוודא שגם דליפת זיכרון של Baileys לאורך זמן לא תפיל את השרת, ויפעל אוטומטית בכל `boot`.

שלב 4 חלופי – מסלול B: עם Claude Code

אם רוצים לדלג את כל המעלה ולתת ל-AI לעשות:

■ B.1 – התקנת Claude Code על המחשב שלכם

BASH

```
npm install -g @anthropic-ai/claude-code  
claude login
```

■ B.2 – תכנסו לשרת שלכם דרך Claude

עיון במצב הזה כשני טרמינלים: אחד SSH לשרת, השני Claude Code על המחשב שלכם.

על המחשב המקומי:

BASH

```
mkdir wa-hub-prep && cd wa-hub-prep  
claude
```

ובתוך Claude הקלידו:

```
המטרה שלי: על שרת Hetzner חדש (Ubuntu 24.04 ARM, IP=<X.X.X.X>),  
להתקין את הפרויקט https://github.com/noamnissan/wa-hub-demo, להריץ אותו  
כ-systemd service תחת משתמש wahub, ולהקים Cloudflare Quick Tunnel  
שיחשוף אותו לאינטרנט.  
  
תעבוד עם Bash דרך ssh root@<X.X.X.X>. תאשר איתי כל שלב לפני שאתה מבצע.  
תתחיל באודיט מצב השרת.
```

Claude Code יעבור צעד-צעד, ישאל לאישור לפעולות risky, וידפיס בסוף את ה-token וה-URL הציבורי. בערך 10 דקות.

מאיפה Claude יודע איך? הוא קורא את הקוד והמדריך הזה. הוא לא מנחש – הוא רואה את אותם השלבים שאתם רואים, ומבצע אותם.

שלב 5 – פיירינג WhatsApp

ה-Hub רץ, אבל הוא עוד לא מחובר לאף מספר. בואו נחבר.

■ 5.1 – שליפת ה-QR מהשרת

BASH

```
# על השרת
TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)

# מצב נוכחי (צריך להיות "qr")
curl -sS -H "Authorization: Bearer $TOKEN" http://127.0.0.1:3060/api/instance/status

# שמרו את ה-QR כתמונה
curl -sS -H "Authorization: Bearer $TOKEN" \
  http://127.0.0.1:3060/api/instance/qr.png > /tmp/qr.png
```

■ 5.2 – העברת ה-QR למחשב המקומי לסריקה

במחשב המקומי (לא בשרת!), הורידו את הקובץ:

BASH

```
# Mac / Linux:
scp -i ~/.ssh/id_ed25519 root@<IP>:/tmp/qr.png ~/qr.png
open ~/qr.png

# Windows PowerShell:
scp -i $HOME\.ssh\id_ed25519 root@<IP>:/tmp/qr.png $HOME\qr.png
Start-Process $HOME\qr.png
```

שימו לב ל-i -: אנחנו נעלנו את SSH ל-key only ב-A.2. אם המפתח שלכם לא בנתיב ברירת המחדל (`~/.ssh/id_ed25519`), תצטרכו לציין מפורש איזה key. אם המפתח כן בברירת מחדל, אפשר להשמיט `-i`.

■ 5.3 – סריקה מהטלפון

1. WhatsApp → הגדרות → מכשירים מקושרים

2. קישור מכשיר → סרקו את הקוד

3. תוך 2-3 שניות – לוג השרת יראה `WhatsApp connected`

טיפ לזיכרון: הריצו

```
curl -sS -H 'Authorization: Bearer $TOKEN' http://127.0.0.1:3060/api/instance/status"
```

כדי לראות בזמן אמת איך ה-state עובר מ-`qr` ל-`connecting` ל-`connected`.

■ 5.4 – בדיקה: שולחים הודעה לעצמכם

BASH

```
TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)

curl -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"to":"972501234567","text":"Hello from my Hub!"}' \
  http://127.0.0.1:3060/api/messages/send/text
```

(החליפו את `972501234567` במספר שלכם, **בלי** `+`, עם קוד מדינה.)

שלב 6 – חשיפה לאינטרנט עם Cloudflare Tunnel

עד עכשיו ה-API רץ רק מקומית (127.0.0.1) / Base44. כל אפליקציה חיצונית לא יכולה להגיע. בואו ננתב את זה החוצה – בלי לפתוח פורט בכלל.

6.1 – התקנת cloudflared

BASH

```
ARCH=$(dpkg --print-architecture) # אם בחרתם arm64 CAX11
curl -fsSL -o cloudflared.deb \
  "https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-
  linux-$ARCH.deb"
dpkg -i cloudflared.deb
cloudflared --version
```

6.2 – Quick Tunnel (URL רנדומלי, ללא חשבון)

הכי קל לדמו:

BASH

```
cloudflared tunnel --url http://127.0.0.1:3060
```

תוך 5 שניות תראו:

```
Your quick Tunnel has been created! Visit it at (it may take some time to be reachable):
https://soft-clouds-rapidly-eat.trycloudflare.com
```

זה ה-URL הציבורי שלכם. בדקו ממחשב אחר:

BASH

```
curl https://soft-clouds-rapidly-eat.trycloudflare.com/healthz
```

חיסרון Quick Tunnel: ה-URL רנדומלי ומשתנה בכל restart. אם השרת נופל ועולה – צריך לעדכן את ה-URL בכל מקום שמשתמש בו. **מצוין לדמו**, פחות לפרודקשן.

■ 6.3 Quick Tunnel כ-`systemd service` (לא ייפול)

כדי שה-Tunnel יישאר חי גם אחרי שהטרמינל נסגר:

BASH

```
cat > /etc/systemd/system/cloudflared-wahub.service <<'EOF'
[Unit]
Description=Cloudflare Quick Tunnel for wa-hub
After=network-online.target wa-hub.service
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/bin/cloudflared tunnel --no-autoupdate --url http://127.0.0.1:3060
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal
DynamicUser=yes

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now cloudflared-wahub.service

# המתינו 6 שניות, אז שלפו את ה-URL מהלוג:
sleep 6
journalctl -u cloudflared-wahub.service -n 30 --no-pager | \
  grep -Eo 'https://[a-z0-9-]+\.\trycloudflare\.com' | head -1
```

■ 6.4 – Named Tunnel (URL קבוע, דורש חשבון Cloudflare + דומיין)

לפרודקשן, השתמשו ב-Named Tunnel:

BASH

```
# התחברות (יפתח URL להעתיק לדפדפן)
cloudflared tunnel login

# יצירת tunnel
cloudflared tunnel create wa-hub

# קונפיגורציה – שמרו את ה-TUNNEL_ID שהודפס למעלה
mkdir -p /etc/cloudflared
cat > /etc/cloudflared/config.yml <<'EOF'
tunnel: <TUNNEL_ID_HERE>
credentials-file: /root/.cloudflared/<TUNNEL_ID_HERE>.json
ingress:
  - hostname: api.yourdomain.com
    service: http://127.0.0.1:3060
  - service: http_status:404
EOF

# DNS routing
cloudflared tunnel route dns wa-hub api.yourdomain.com

# התקנה כ-systemd
cloudflared service install
systemctl enable --now cloudflared
```

עכשיו ה-API שלכם זמין ב- <https://api.yourdomain.com> באופן קבוע, מאחורי Cloudflare (DDoS protection חנימ, SSL אוטומטי).

שלב 7 – שימוש מכל מערכת

ה-API שלך הוא **REST + JSON + Bearer auth** – הסטנדרט הכי משעמם בעולם. וזה היתרון. כל פלטפורמה שיודעת לעשות HTTP request מסוגלת לדבר איתו.

בכל הדוגמאות:

- ה-URL הציבורי של ה-Tunnel שלך = `HUB_URL`
- ה-Bearer מ-`.env` = `HUB_TOKEN`
- שמור אותם **רק בצד שרת / סודות**, לעולם לא ב-JS של דפדפן

■ 7.1 – Bubble

ב-Bubble Plugins → **API Connector**:

1. **Add another API** → תן שם `WhatsApp Hub`

2. **Authentication**: `Private key in header`

3. **Key name**: `Authorization` · **Key value**: `Bearer`

4. **Add call**

◦ **Name**: `Send Text`

◦ **Method**: `POST`

◦ **URL**: `/api/messages/send/text`

◦ **Body type**: `JSON`

◦ **Body**: `{"to":"","text":""}`

◦ סמן את `to` ו-`text` כ-parameters

ב-workflow: `When Button "Send" is clicked → API call WhatsApp Hub Send Text`

JAVASCRIPT

```

// functions/index.js
const { onRequest } = require("firebase-functions/v2/https");
const { defineSecret } = require("firebase-functions/params");

const HUB_TOKEN = defineSecret("HUB_TOKEN");
const HUB_URL = "https://your-tunnel.trycloudflare.com";

exports.sendWa = onRequest({ secrets: [HUB_TOKEN] }, async (req, res) => {
  const r = await fetch(`${HUB_URL}/api/messages/send/text`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${HUB_TOKEN.value()}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify(req.body),
  });
  res.status(r.status).json(await r.json());
});

```

הזריקו את הסוד: `firebase functions:secrets:set HUB_TOKEN`

Make / n8n / Zapier – 7.3 ■

שליחה:

1. הוסיפו (n8n) HTTP Node / (Make) HTTP Request module / Webhooks by Zapier
2. URL: /api/messages/send/text
3. Method: POST
4. Headers: Authorization: Bearer + Content-Type: application/json
5. Body: {"to":"+972...", "text":"hi"}

קבלת webhook:

1. צרו Webhook ב-Make/n8n/Zapier → תקבלו URL

2. רשמו אותו ב-Hub:

BASH

```
curl -X PUT -H "Authorization: Bearer $HUB_TOKEN" \
-H "Content-Type: application/json" \
-d '{"url":"<your make/n8n webhook url>","events":["message.incoming"]}' \
$HUB_URL/api/instance/webhook
```

כל הודעה נכנסת מופיעה בתוך n8n/Make כטריגר אוטומטי.

Node.js / Next.js – 7.4 ■

JAVASCRIPT

```
// app/api/send-wa/route.js (Next.js App Router)
export async function POST(req) {
  const { to, text } = await req.json();
  const r = await fetch(`${process.env.HUB_URL}/api/messages/send/text`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${process.env.HUB_TOKEN}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ to, text }),
  });
  return Response.json(await r.json(), { status: r.status });
}
```


PYTHON

```
import os, requests

HUB_URL = os.environ["HUB_URL"]
HUB_TOKEN = os.environ["HUB_TOKEN"]

def send_whatsapp(to: str, text: str):
    r = requests.post(
        f"{HUB_URL}/api/messages/send/text",
        headers={"Authorization": f"Bearer {HUB_TOKEN}"},
        json={"to": to, "text": text},
        timeout=15,
    )
    r.raise_for_status()
    return r.json()
```

:webhook (FastAPI) קבלת

PYTHON

```
import hmac, hashlib, os
from fastapi import FastAPI, Request, HTTPException

app = FastAPI()
SECRET = os.environ["HUB_WEBHOOK_SECRET"]

@app.post("/wa/incoming")
async def incoming(req: Request):
    body = await req.body()
    got = req.headers.get("x-hub-signature", "")
    want = "sha256=" + hmac.new(SECRET.encode(), body, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(got, want):
        raise HTTPException(401)
    event = await req.json()
    # ... your logic ...
    return {"ok": True}
```

PHP

```
use Illuminate\Support\Facades\Http;

$r = Http::withToken(env('HUB_TOKEN'))
    ->post(env('HUB_URL') . '/api/messages/send/text', [
        'to' => $to,
        'text' => $text,
    ]);
return response()->json($r->json(), $r->status());
```

Google Sheets / Apps Script — 7.7 ■

JAVASCRIPT

```
function sendWhatsApp(to, text) {
    const HUB_URL = PropertiesService.getScriptProperties().getProperty('HUB_URL');
    const HUB_TOKEN = PropertiesService.getScriptProperties().getProperty('HUB_TOKEN');
    return UrlFetchApp.fetch(`${HUB_URL}/api/messages/send/text`, {
        method: 'post',
        contentType: 'application/json',
        headers: { Authorization: `Bearer ${HUB_TOKEN}` },
        payload: JSON.stringify({ to, text }),
    }).getContentText();
}

:sheet-ב שורה לכל הודעה //
function bulkSend() {
    const rows = SpreadsheetApp.getActiveSheet().getDataRange().getValues();
    rows.slice(1).forEach(([phone, message]) => sendWhatsApp(phone, message));
}
```

BASH

```
# /etc/cron.daily/wa-summary.sh
TOKEN=$(cat /etc/hub.token)
curl -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"to":"+972501234567","text":"Daily summary ready"}' \
  https://your-tunnel.trycloudflare.com/api/messages/send/text
```

התובנה: כמעט כל פלטפורמת no-code/low-code היום יודעת לעשות HTTP. ה-Hub שלך הופך לכל אחת מהן ל"ספק WhatsApp" באופן מיידי, בלי לקנות עוד תוסף או תשלום per-message.

שלב 8 – אינטגרציה מלאה ל-Base44

Base44 מקבל פרק מורחב כי הוא נפוץ מאוד בקהילה של no-code/low-code, ויש בו כמה גיצ'ות אמיתיות שכדאי לדעת לפני שמתחילים.

■ 8.1 – יצירת פרויקט וחיבור CLI

BASH

```
# על המחשב המקומי
npm install -g base44
mkdir wa-chat && cd wa-chat
npx base44 login # פותח דפדפן לאישור
npx base44 create wa-chat -p . -t backend-and-client
```

זה יוצר פרויקט Vite + React + Tailwind עם תיקיית `base44/` שמכילה את הגדרות ה-entities, functions, ו-`.config`.

■ 8.2 – הזרקת סודות

BASH

```
npx base44 secrets set \
  WA_HUB_URL=https://your-tunnel.trycloudflare.com \
  WA_HUB_TOKEN=<HUB_TOKEN_from_env> \
  WA_HUB_SECRET=<WEBHOOK_SECRET_from_env> \
  CHAT_NUMBER=972585802298
```

הסודות זמינים בתוך פונקציות backend דרך `Deno.env.get(...)`.

■ 8.3 Entity להודעות (זהירות מ-RLS!)

JSONC

```
// base44/entities/message.jsonc
{
  "name": "Message",
  "type": "object",
  "properties": {
    "direction": { "type": "string", "enum": ["incoming", "outgoing"] },
    "text": { "type": "string" },
    "ts": { "type": "number" },
    "external_id": { "type": "string" },
    "chat_number": { "type": "string" }
  },
  "required": ["direction", "text", "ts", "chat_number"],
  "rls": {
    "create": true,
    "read": true,
    "update": true,
    "delete": true
  }
}
```

גיצ'ה #1 – RLS: Base44 entities **ברירת מחדל owner-only**. אם תשמיט את בלוק `rls`, רק המשתמש שיצר את הרשומה יוכל לראות אותה. במקרה של webhook (אנונימי לחלוטין) – `Message.create()` יעבוד דרך `asServiceRole`, אבל `Message.list()` מהפרונט (גם של אדמין מחובר) יחזיר 401. הפתרון: או להגדיר `"rls": { read: true, ... }` כמו למעלה, או להעטוף את כל קריאות ה-entity בפונקציות עם `service role`.

TYPESCRIPT

```
// base44/functions/send-message/index.ts
import { createClientFromRequest } from "npm:@base44/sdk";

const HUB_URL      = Deno.env.get("WA_HUB_URL")!;
const HUB_TOKEN    = Deno.env.get("WA_HUB_TOKEN")!;
const CHAT_NUMBER  = Deno.env.get("CHAT_NUMBER")!;

Deno.serve(async (req) => {
  try {
    const { text } = await req.json();
    if (!text?.trim()) {
      return Response.json({ error: "text_required" }, { status: 400 });
    }

    const r = await fetch(`${HUB_URL}/api/messages/send/text`, {
      method: "POST",
      headers: {
        Authorization: `Bearer ${HUB_TOKEN}`,
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ to: CHAT_NUMBER, text }),
    });

    const data = await r.json();
    if (!r.ok) return Response.json({ error: "hub_error", details: data }, { status: 502 });

    // Mirror locally so the UI shows the sent message immediately.
    const base44 = createClientFromRequest(req);
    await base44.asServiceRole.entities.Message.create({
      direction: "outgoing",
      text,
      ts: Date.now(),
      external_id: data.id,
      chat_number: CHAT_NUMBER,
    });

    return Response.json({ ok: true, id: data.id });
  } catch (err) {
    return Response.json({ error: "internal", message: String(err) }, { status: 500 });
  }
});
```

TYPESCRIPT

```
// base44/functions/whatsapp-webhook/index.ts
import { createClientFromRequest } from "npm:@base44/sdk";
import { createHmac, timingSafeEqual } from "node:crypto";
import { Buffer } from "node:buffer";

const WEBHOOK_SECRET = Deno.env.get("WA_HUB_SECRET")!;
const CHAT_NUMBER = Deno.env.get("CHAT_NUMBER")!;

Deno.serve(async (req) => {
  if (req.method !== "POST") return new Response("method not allowed", { status: 405 });

  const body = await req.text(); // bytes – must not parse first
  const given = req.headers.get("x-hub-signature") || "";
  const want = "sha256=" + createHmac("sha256", WEBHOOK_SECRET)
    .update(body).digest("hex");

  // Constant-time compare – important to prevent timing attacks.
  if (given.length !== want.length) return new Response("bad sig", { status: 401 });
  if (!timingSafeEqual(Buffer.from(given), Buffer.from(want))) {
    return new Response("bad sig", { status: 401 });
  }

  const event = JSON.parse(body);

  if (event.event === "message.incoming" && event.data?.type === "text") {
    const base44 = createClientFromRequest(req);
    await base44.asServiceRole.entities.Message.create({
      direction: "incoming",
      text: event.data.text || "",
      ts: event.data.timestamp || Date.now(),
      external_id: event.data.id || "",
      chat_number: CHAT_NUMBER,
    });
  }

  return new Response("ok");
});
```

למה `node:crypto` ולא `Web Crypto`? Base44 functions רצים על Deno עם Node compatibility

`node:crypto` layer. נתמך מלא ועובד בלי `polyfill`.

אם אתם פורסים על סביבה אחרת (Cloudflare Workers, Vercel Edge) שלא תומכת ב-Node modules, השתמשו ב-Web Crypto:

TYPESCRIPT

```
const enc = new TextEncoder();
const key = await crypto.subtle.importKey(
  "raw", enc.encode(SECRET),
  { name: "HMAC", hash: "SHA-256" }, false, ["sign"]
);
const sig = await crypto.subtle.sign("HMAC", key, enc.encode(body));
const hex = [...new Uint8Array(sig)].map(b => b.toString(16).padStart(2, "0")).join("");
const want = "sha256=" + hex;
```

8.6 – פונקציה: שליפת הודעות (לפרונט)

TYPESCRIPT

```
// base44/functions/list-messages/index.ts
import { createClientFromRequest } from "npm:@base44/sdk";

const CHAT_NUMBER = Deno.env.get("CHAT_NUMBER")!;

Deno.serve(async (req) => {
  try {
    const base44 = createClientFromRequest(req);
    const all = await base44.asServiceRole.entities.Message.list("ts", 200);
    const filtered = (all || []).filter((m: any) => m.chat_number === CHAT_NUMBER);
    return Response.json({ messages: filtered });
  } catch (err) {
    return Response.json({ error: "internal", message: String(err) }, { status: 500 });
  }
});
```


8.7 – קוד פרונט: אבל יש wrapper

ב-Frontend, כשקוראים לפונקציה – חשוב לדעת:

JAVASCRIPT

```
// ✅ נכון
const r = await base44.functions.invoke("list-messages", {});
setMessages(r?.data?.messages || []);

// ❌ לא נכון – לא יעבוד
const r = await base44.functions.invoke("list-messages", {});
setMessages(r?.messages || []);
```

גי'צ'ה #2 – Axios wrapper: `base44.functions.invoke()` מחזיר אובייקט בסגנון `axios`:

`{ data, status, headers, ... }`. הקריאה האמיתית למה שהפונקציה החזירה היא דרך `.data`.
אם אתם רואים OK 200 בנטוורק והקוד מתנהג כאילו הוא נכשל – זה כנראה זה.

8.8 – רישום ה-webhook ב-Hub אחרי פריסה

אחרי `npx base44 deploy -y`, הפרויקט שלכם מקבל URL ציבורי. רשמו אותו ב-Hub:

BASH

```
TOKEN=<HUB_TOKEN>
WEBHOOK="https://wa-chat-XXXXXXX.base44.app/api/functions/whatsapp-webhook"

curl -X PUT -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"url":"'${WEBHOOK}',"events":["message.incoming"]}' \
  https://your-tunnel.trycloudflare.com/api/instance/webhook
```

גי'צ'ה #3 – Webhook לא persistent: הקריאה ל-`PUT /api/instance/webhook` שומרת את ה-URL ב-**זיכרון בלבד**. בכל restart של ה-Hub (תקלה, עדכון, reboot) נמחק וה-webhook יפסיק לעבוד עד שתרשמו שוב.

הפתרון: הזריקו את ה-URL ל-`.env` של ה-Hub כדי שיטען בכל boot:

BASH

```
sed -i "s|^WEBHOOK_URL=.*|WEBHOOK_URL=$WEBHOOK|" /srv/wa-hub-demo/.env
sed -i "s|^WEBHOOK_EVENTS=.*|WEBHOOK_EVENTS=message.incoming|" /srv/wa-hub-demo/.env
systemctl restart wa-hub
```

שלב 9 – אבטחה לפרודקשן

לפני שמשחררים את זה לעולם:

בדיקה	פעולה
HTTPS בלבד?	אם אתם משתמשים ב-Tunnel – כן, אוטומטית
Token חזק?	<code>openssl rand -hex 32</code> נותן 256 ביט – מספיק
?Rate limit	בקובץ <code>.env</code> : <code>RATE_LIMIT_PER_MIN=120</code> . עדכנו לפי הצורך
Webhook ?signed	הקוד מסמן HMAC-SHA256. הצד השני חייב לאמת
השרת מתעדכן?	<code>apt-get install -y unattended-upgrades</code>
ניטור?	<code>/healthz</code> על Uptime Robot + <code>journalctl -u wa-hub -f</code>
גיבוי?	תקיית <code>/srv/wa-hub-demo/data/auth</code> מחזיקה את ה-session. ל-S3/R2 פעם ביום <code>rsync</code>
סודות לא ב-git?	<code>.gitignore</code> .כבר חוסם <code>.env</code> . וודאו שלא הוספתם בטעות
תקרת זיכרון?	הוא יחזיר את השירות אוטומטית OOM-כולל <code>MemoryMax=512M</code> . ב <code>systemd unit</code>

9.1 – סיבוב Token

אם ה-Token דלף או חשדתם:

BASH

```
NEW=$(openssl rand -hex 32)
sed -i "s/^HUB_TOKEN=.* /HUB_TOKEN=$NEW/" /srv/wa-hub-demo/.env
systemctl restart wa-hub
echo "$NEW"
```

עדכנו ב-Base44 secrets (או היכן שאתם משתמשים) – וזהו. 30 שניות downtime.

■ 9.2 – זיכרון לטווח ארוך

Baileys שומר היסטוריית chats ב-RAM. ב-instance עם מעט תעבורה (פחות מ-100 הודעות/יום) זה לא בעיה. ב-instance פעיל יותר, הזיכרון עלול לטפס.

הגנות שכבר במקום:

- `syncFullHistory: false` ב- `src/baileys/socket.js` – אנחנו לא מורידים את כל ההיסטוריה הישנה
- `markOnlineOnConnect: false` – חוסכים traffic של "אונליין" notifications
- `MemoryMax=512M` ב- `systemd unit` – אם בכל זאת זה תופח מעבר, `systemd` יחזיר את השירות (Baileys) (ייחבר מחדש)

אם יש לכם volume גבוה במיוחד (אלפי הודעות ביום): שקלו להפחית את ה-cache ב-Baileys, או לפצל ל-`multiple instances`.

גיצ'ות מהשטח – דברים שלא תמצאו בתיעוד

■ #1 Baileys 7 ומספרי LID

החל מ-2025, WhatsApp מציגים **LID (Logical ID)** – מזהה ייחודי לכל משתמש שלא חושף את מספר הטלפון שלו. זה נועד למנוע harvesting של מספרים על-ידי mass-scraping.

```
fromNumber = "972501234567" ← phone: לפני:
fromNumber = "8362502693023" ← LID: אחרי: מספר 13 ספרות מוזר
```

ה-Hub מטפל בזה אוטומטית: קוד `extractPhone()` מנסה לחלץ מספר טלפון אמיתי דרך `senderPn` → `participantPn` → `remoteJidAlt`. אם אין – חוזר ל-LID. שדה `fromLid: true/false` מאותת לכם איזה הזיהוי המקורי.

אם אתם מסננים לפי `fromNumber` ורואים שמסננים יותר מדי – בדקו `fromLid` ב-payload.

■ #2 Base44 functions.invoke מחזיר axios wrapper

JAVASCRIPT

```
const r = await base44.functions.invoke("foo", {});
// r = { data: <actual response>, status: 200, headers: {...} }
```

תקראו `r.data.X` ולא `r.X`. אם הקוד שלכם מתנהג כאילו פונקציה נכשלת אבל ה-network tab מראה OK 200 – זה הסיבה.

■ #3 Base44 entities ברירת מחדל owner-only

ללא בלוק `rls` בסכמה, רק היוצר רואה את הרשומה. webhooks ש-anon נכנסות לא יוכלו לקרוא, גם דרך `asServiceRole` במצב מסוים. הגדירו `rls: { read: true, create: true, ... }` או עברו את כל הקריאות דרך functions עם service role.

■ #4 Webhook URL נמחק ב-restart

`PUT /api/instance/webhook` שומר ב-זיכרון בלבד. שמרו ב-`.env`:

BASH

```
WEBHOOK_URL=https://your-receiver.com/wa
WEBHOOK_EVENTS=message.incoming
```

■ #5 – Quick Tunnel URL מתחדש בכל restart

מצוין לדמו, אסון לפרודקשן. עברו ל-Named Tunnel עם דומיין משלכם ברגע שאתם עוברים מ-experiment ל-production.

■ #6 – 14 Linked Device timeout יום

אם הטלפון הראשי לא היה online במשך 14 יום, WhatsApp מנתקים את כל ה-Linked Devices. תצטרכו פייר QR מחדש. אם אתם רוצים שזה יחזיק יותר – ודאו שהטלפון הראשי מחובר לרשת לפחות פעם ב-13 יום.

■ #7 – Windows + Node.js 24 + נתיב עברי = bug ב-base44 0.0.52

על Windows עם שם משתמש עברי (כמו "נעם ניסן"), הגרסה האחרונה של חבילת `base44` ב-npm נכשלת ב-`cp` של `assets`. הפתרון:

BASH

```
npm install --save-dev base44@0.0.51
```

נהוג להתקין global גם:

BASH

```
npm install -g base44@0.0.51
```

(זה ייעלם כשהם יתקנו את ה-bug, אבל נכון לכתיבת המדריך הזה הבעיה קיימת.)

■ #8 – apt-get zombie על שרתי cloud

ראיתי שרת Hetzner שבו `apt-get update` נשאר רץ במשך **11 ימים**, חוסם את כל ניסיונות העדכון של `unattended-upgrades`. אם עדכוני אבטחה לא תופסים – בדקו:

BASH

```
ps -ef | grep apt-get | grep -v grep
# אם רואים תהליך מ-3+ ימים – הרגו:
kill -9 <PID>
rm -f /var/lib/dpkg/lock-frontend /var/lib/dpkg/lock /var/cache/apt/archives/lock
apt-get update
```

פתרון בעיות נפוצות

השירות מופעל אבל QR לא מופיע ▼

BASH

```
journalctl -u wa-hub -n 50 --no-pager
```

חפשו `QR generated` . אם לא מופיע – בדקו שיש חיבור לאינטרנט:

BASH

```
curl -sS https://web.whatsapp.com | head -1
```

"already paired" אבל בטלפון לא רואים את ההתקן ▼

ה-Hub חושב שהוא מחובר אבל בעצם לא. אפסו:

BASH

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
http://127.0.0.1:3060/api/instance/logout
```

ובקשו QR חדש.

הודעות לא נשלחות – not_connected ▼

ב-95% מהמקרים זה כי הטלפון לא היה מחובר לאינטרנט יותר מ-14 יום, וה-session פג. הריצו את תהליך הפיירינג מחדש מהשלב 5.

Cloudflare Tunnel נופל ▾

BASH

```
systemctl status cloudflared-wahub # Quick tunnel
systemctl status cloudflared       # Named tunnel
journalctl -u cloudflared-wahub -n 50 --no-pager
```

לרוב זה תזוזה ב-URL (ב-quick) – restart ייצור URL חדש. שמרו את ה-URL החדש ועדכנו בכל מקום שהשתמשתם בו.

Webhook לא נכנס ל-Base44 (OK 200 בלוג של Hub, אבל ה-entity ריק) ▾

1. ראו את הלוגים של הפונקציה: `npx base44 logs --function whatsapp-webhook --limit 20`
2. אם רואים שגיאת RLS – וודאו שב- `message.jsonc` יש

```
rls: { create: true, read: true, ... }
```
3. אם רואים שגיאת חתימה – בדקו ש- `WA_HUB_SECRET` ב-Base44 secrets זהה לחלוטין ל- `WEBHOOK_SECRET` ב- `.env` של השרת

הודעות נכנסות יש, אבל fromNumber הוא LID ▾

ראו "גיצ'ה #1" למעלה. השדה `fromLid: true` מאותת שזה LID. אם רוצים את מספר הטלפון האמיתי – אין דרך, WhatsApp לא חושפים את זה. תזהו לקוחות לפי `chat` (ה-JID של השיחה) שיציב לאורך זמן.

הצעדים הבאים

- **רוצים multi-tenant?** הוסיפו טבלת tenants ל-DB → לכל לקוח `instance_id` ו-port נפרד. אפשרות נוספת: להריץ `wa-hub-demo` כמה פעמים בקונטיינרים על אותו שרת.
- **רוצים AI אוטומטי?** הוסיפו פונקציה שמקבלת `message.incoming`, שולחת ל-Claude/GPT, ומחזירה תגובה חזרה דרך `POST /api/messages/send/text`. צ'אטבוט תוך 30 שורות.
- **רוצים התראות לצוות?** ב- `/api/instance/webhook` אפשר להגדיר Slack/Discord webhook במקום (או בנוסף ל-) Base44.
- **רוצים dashboard?** ה-WebSocket ב- `:3061` משדר כל אירוע בזמן אמת – חברו לאפליקציית React/Vue/Svelte.
- **רוצים סיכומים יומיים?** `cron` + `curl` ב- `/etc/cron.daily/`.

כלים שנדאי להכיר

- [Baileys](#) – הספרייה שעושה את הקסם של WhatsApp Web
 - [Cloudflared](#) – Tunnel
 - [Express](#) – REST framework
 - [Hetzner Cloud](#) – VPS מומלץ
 - [Uptime Robot](#) – חינום לניטור 50 endpoints
 - [Cloudflare R2](#) – אחסון זול לגיבוי `data/auth`
-

שאלות נפוצות

- Q: זה חוקי? A:** כן. WhatsApp לא אוסר על Linked Devices לא-רשמיים, אבל שומר לעצמו את הזכות לחסום מספרים שמתנהגים כספאם. שלחו רק להודעות שביקשו מכם.
- Q: השרת יעמוד בעומס? A:** CAX11 (4GB RAM) טוב לעד ~50 הודעות לשנייה. אם אתם מצפים ליותר – שדרגו ל-CAX21 (8GB, €7.59), או הריצו מספר instances.
- Q: מה קורה אם Hetzner נופל? A:** ב-12 חודשים אחרונים – uptime ~99.95%. לפרודקשן רצינית שקלו multi-region או fallback ל-cloud אחר.
- Q: מה הסיכון שיחסמו לי את המספר? A:** נמוך אם אתם משתמשים בו כמו אדם רגיל – תגובות, סיכומים, לא יותר מ-1000 הודעות ביום למספרים אחרים. גבוה אם אתם שולחים cold outreach.
- Q: יש Web Crypto במקום node:crypto? A:** כן – ראו דוגמה ב-8.5 לסביבות שלא תומכות ב-Node modules (Cloudflare Workers, Vercel Edge וכו'). שני הפתרונות עובדים זהה לחתימת HMAC.
- Q: אפשר להשתמש בקוד מסחרי? A:** כן. רישיון MIT – תפרק, תשנה, תפרסם, תמכור. אם תרגישו בנוח, תזכירו את הפרויקט המקורי.

יש לכם עכשיו תשתית WhatsApp עצמאית מקצועית.

קוד פתוח: github.com/noamnissan/wa-hub-demo